



Reporte de Análisis Estático de Código

Integrantes (Programangos):

Carlos Daniel Albarracín Cruz

Pablo Andrés Pérez Beltrán

David Nickolai Parra Ariza

Jhoan Sebastián Yaya García

Docente: Oscar Eduardo Álvarez Rodríguez

Fecha: 14/06/2026

Curso:

2016701: Ingeniería de Software I

Facultad de Ingeniería, Universidad Nacional de Colombia

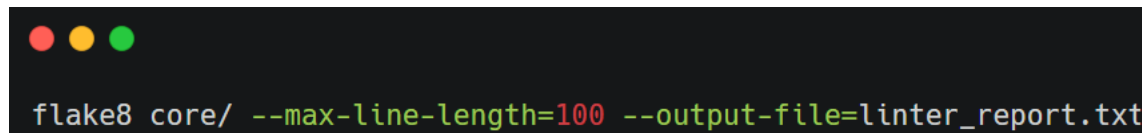
1. Herramienta utilizada

La herramienta seleccionada para el análisis estático del código fue **flake8**, un linter ampliamente adoptado en el ecosistema Python que combina tres verificadores en uno: pyflakes (detección de errores lógicos como imports no usados), pycodestyle (conformidad con la guía de estilo PEP 8) y mccabe (complejidad ciclomática).

Se eligió flake8 por su simplicidad de uso, su integración directa con proyectos Django y su capacidad de generar reportes en archivo de texto, lo cual facilita su revisión y documentación. Adicionalmente, es compatible con el flujo de trabajo del equipo al ejecutarse desde la línea de comandos sin configuración adicional compleja.

2. Configuración aplicada

El linter fue ejecutado sobre la carpeta **core/** del proyecto backend, que contiene toda la lógica de la aplicación (domain, infra, services, controllers, serializers y tests). Se aplicó la siguiente configuración personalizada:



```
flake8 core/ --max-line-length=100 --output-file=linter_report.txt
```

La única regla personalizada fue `--max-line-length=100`, que amplía el límite predeterminado de 79 caracteres a 100. Esta decisión se tomó porque el proyecto utiliza nombres descriptivos en los modelos y serializers de Django REST Framework, donde líneas de hasta 100 caracteres son comunes y aceptables sin afectar la legibilidad.

El resultado del reporte fue redirigido al archivo **linter_report.txt** dentro de `src/` para su revisión y documentación.

3. Resultados obtenidos antes de la corrección

La primera ejecución del linter arrojó 16 hallazgos distribuidos en cuatro tipos de error:

Código	Tipo	Cantidad	Archivos afectados
E501	Línea muy larga (> 100 caracteres)	8	procedures_controller.py, user_serializer.py, urls.py
E302	Faltan dos líneas en blanco antes de clase/función	5	role_controller.py, role.py, role_repository.py,

Código	Tipo	Cantidad	Archivos afectados
			role_serializer.py, role_service.py
W391	Línea en blanco al final del archivo	1	notice.py
F401	Import no utilizado	2	models.py, views.py

- Los errores E302 se concentraron en los archivos del módulo *role*, que corresponden al tutorial ‘Hola Mundo’ de la entrega anterior y no seguían completamente la guía de estilo PEP 8.
- Los errores E501 se debieron a líneas largas en el serializer de usuarios y en el archivo de URLs.
- El error W391 fue una línea en blanco residual al final de *notice.py*
- Los F401 correspondían a imports generados automáticamente por Django al crear la app (*models.py* y *views.py*) que no fueron utilizados.

Figura 1. Captura de pantalla del reporte de errores generado por flake8 (16 hallazgos)

Reporte generado en <code>src/linter_report.txt</code> — 16 hallazgos:			
Tipo	Cantidad	Descripción	
E501 (línea muy larga)	8	<code>procedures_controller.py, user_serializer.py, urls.py</code>	
E302 (faltan blank lines)	5	<code>role_controller.py, role.py, role_repository.py, role_serializer.py, role_service.py</code>	
W391 (blank line al final)	1	<code>notice.py</code>	
F401 (import no usado)	2	<code>models.py, views.py</code>	

4. Correcciones aplicadas

Una vez identificados los errores, se aplicaron las siguientes correcciones sobre los archivos afectados:

- **E501 - Líneas muy largas**

En *procedures_controller.py* y *user_serializer.py* se dividieron las líneas largas usando continuación implícita dentro de paréntesis. En *urls.py* se reorganizaron los imports en múltiples líneas.

- **E302 - Líneas en blanco faltantes**

En los archivos del módulo role (*role_controller.py*, *role.py*, *role_repository.py*, *role_serializer.py*, *role_service.py*) se agregaron dos líneas en blanco antes de cada definición de clase y función a nivel de módulo, según lo establece PEP 8.

- **W391 - Línea en blanco al final**

En *notice.py* se eliminó la línea en blanco residual al final del archivo.

- **F401 - Imports no utilizados**

En *models.py* y *views.py* se eliminaron los imports generados automáticamente por Django que no estaban siendo utilizados en el proyecto.

5. Resultado tras la corrección

Tras aplicar todas las correcciones, se ejecutó nuevamente el linter con la misma configuración. El reporte generado no arrojó ningún hallazgo, confirmando que el código cumple con los estándares de estilo definidos:



```
$ flake8 core/ --max-line-length=100 --output-file=linter_report.txt
SIN ERRORES
```

Figura 2. Captura de pantalla de la ejecución del comando con resultado SIN ERRORES

```
$ Set-Location -LiteralPath "C:\Users\nicko\SISA\project-backend\src"; if ($?) { Remove-Item -LiteralPath "linter_report.txt" -ErrorAction SilentlyContinue; flake8 core/ --max-line-length=100 --output-file=linter_report.txt; Get-Content -LiteralPath "linter_report.txt"; if ((Get-Item "linter_report.txt").Length -eq 0) { Write-Host "SIN ERRORES" } }
```

SIN ERRORES